Using Gemini for Coding in Scientific Work

A practical guide to Gemini Chat and Gemini Code Assist

FYS4411 – Reinforcement Learning and Agentic AI

University of Oslo, Department of Physics

2026

This guide introduces the use of Gemini for coding in scientific work. It covers two complementary modes of use: Gemini Chat, used through the Gemini web app for planning, explanation, and problem formulation, and Gemini Code Assist, used inside an IDE such as Visual Studio Code for code completion, code generation, chat, smart actions, and larger code edits. The emphasis is practical and methodological. The goal is not to replace scientific understanding, but to show how these tools can support reproducible programming, testing, debugging, documentation, and exploratory scientific computation.

Contents

1 Motivation

Scientific programming often sits between several forms of expertise. A student or researcher must understand the scientific model, translate it into an algorithm, implement the algorithm correctly, test it on simple cases, and finally interpret numerical output. Mistakes can occur at any stage. A program may run without implementing the intended equation. A simulation may use inconsistent units. A fitting script may optimize a curve while hiding an invalid model assumption. A plot may look reasonable even when the data were processed incorrectly.

Gemini Chat and Gemini Code Assist can be useful in this setting because they reduce the friction between formulation and implementation. They can help draft code, explain library calls, generate tests, review error messages, suggest documentation, and organize small computational projects. Gemini Chat is most useful outside the editor, when the task is conceptual: planning a workflow, comparing approaches, asking for assumptions, or drafting a reproducibility checklist. Gemini Code Assist is most useful inside the editor, where the tool can use the current file, selected code, terminal output, and workspace context.

The tools should not be understood as sources of scientific authority. They are assistants for a workflow. They can make suggestions, but the user remains responsible for correctness. In scientific use, this means that generated code should be inspected, tested, and compared against cases where the answer is known. The practical goal is therefore not automation in the naive sense. The goal is a disciplined human-AI workflow in which the tool accelerates routine programming while the student remains in control of assumptions, validation, and interpretation.

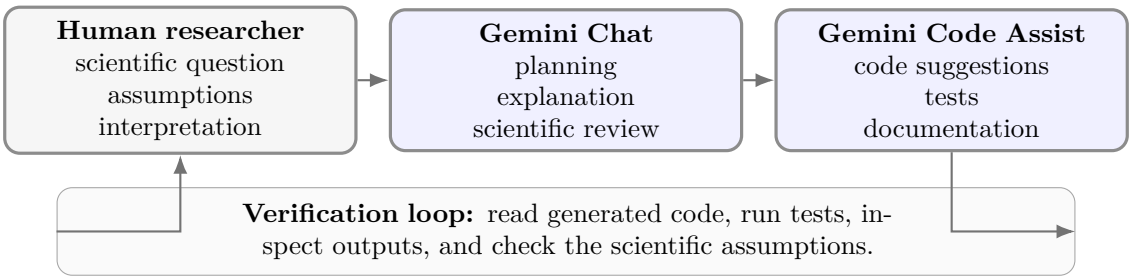

Figure 1: A useful division of labor. Gemini can assist with planning and implementation, but the scientific question and interpretation remain human responsibilities.

2 Two tools: Gemini Chat and Gemini Code Assist

Gemini Chat and Gemini Code Assist are related but not identical. Gemini Chat is a general conversational interface. It is useful for asking questions, exploring ideas, checking explanations, drafting text, and planning code before implementation. According to Google's Gemini Apps documentation, Gemini can be used through the web app, and users can start conversations, attach files or images where supported, edit prompts, and review alternative drafts in some cases (Google Gemini Apps Help, 2026b,a).

Gemini Code Assist is the coding-oriented tool integrated into supported IDEs. The Google documentation describes it as an AI-powered collaborator for generating code, receiving code completions, using smart actions, transforming code, and interacting through chat in VS Code and supported JetBrains IDEs (Google for Developers, 2026i,f,j). It is therefore closer to a

pair-programming assistant than to a general chat interface.

For a scientific project, the two interfaces can be used together. Gemini Chat may be used to formulate the computational experiment: What assumptions are being made? What tests should be run? What plots would reveal model failure? Gemini Code Assist may then be used to implement the functions, inspect selected code, generate tests, and update documentation inside the repository.

	Gemini Chat	**Gemini Code Assist**
Typical location	Web app or conversational interface	IDE such as VS Code
Best use	Planning, explanation, scientific reasoning, prompt iteration	Code completions, selected-code explanation, edits, tests, project-aware assistance
Context	Prompt, conversation, attached files when available	Current file, selected code, terminal output, workspace files and folders where configured
Risk	Plausible but unsupported explanations	Plausible but incorrect code changes
Best safeguard	Ask for assumptions and verification criteria	Review diffs, run tests, inspect generated code

3 Access, installation, and login

3.1 Gemini Chat

For Gemini Chat, the basic workflow is to go to `https://gemini.google.com` and sign in with a Google account if sign-in is required for the desired features. Google's Gemini Apps Help states that some features can be used without signing in, while additional features and saved Gemini Apps activity require a Google account (Google Gemini Apps Help, 2026a). The same documentation states that access can depend on account type, age, country availability, and whether a work or school administrator has enabled the service.

For coding work, Gemini Chat is useful even before opening an IDE. A student can ask for a computational plan, compare possible numerical methods, draft a project structure, or ask for a testing strategy. The result should then be transferred into the editor in small parts, not pasted as an entire unreviewed project.

3.2 Gemini Code Assist in VS Code

Gemini Code Assist requires a supported IDE and an account or subscription with access to the service. The Google documentation describes setup for Gemini Code Assist Standard and Enterprise, and the coding documentation assumes that the relevant IDE setup has already been completed (Google for Developers, 2026j,i). At the time of writing, Google's deprecation notice states that from June 18, 2026, Gemini Code Assist IDE extensions and Gemini CLI stopped serving requests for certain consumer tiers, while Standard and Enterprise access remain unchanged (Google for Developers, 2026g). Students should therefore check the current course, university, or organizational access route before installing.

In VS Code, the practical steps are normally as follows. Install Visual Studio Code if it is not already installed. Open the Extensions view using `Ctrl+Shift+X` on Windows/Linux or `Cmd+Shift+X` on macOS. Search for *Gemini Code Assist* and install the official extension. After installation, sign in from the Gemini Code Assist interface in the activity bar or command palette. Use the same Google account that has access to Gemini Code Assist. Some access routes may require selecting or linking a Google Cloud project (Google Developer Program Help, 2026).

A simple test after login is to open a Python file and write a precise comment:

```
# Compute the fidelity |<psi_target|psi>|^2
# for two normalized complex state vectors.
```

The Code Assist documentation describes a workflow where comments can be used as prompts in code files, followed by a generation shortcut. In VS Code, the documented shortcut for generating code from such a comment is `Control+Enter` on Windows/Linux or `Control+Return` on macOS, with generated code appearing as ghost text that can be accepted with `Tab` (Google for Developers, 2026j). Keybindings may differ depending on extensions and operating system settings, so the important point is to verify that small local completions work before using the tool in a larger project.

3.3 Troubleshooting first setup

The most common setup problems are usually not conceptual. They are account and configuration problems. If the tool does not respond, first check that the correct Google account is signed in, that the account has access to the relevant Gemini Code Assist edition, that the extension is enabled, and that the current file language is supported. Google's documentation also notes known issues such as sign-in timeouts, chat truncation for large files, and Vim-mode interactions with accepting or dismissing suggestions (Google for Developers, 2026j,c).

For a course setting, it is useful to ask students to perform a minimal verification exercise during setup: generate a short function, ask for a docstring, ask the chat to explain the function, and run a small test. This reveals whether the installation is active and whether the student understands how to accept or reject generated suggestions.

4 Using Gemini Chat for coding

Gemini Chat is most useful when the task is not yet ready for code. Scientific programming often begins with an unclear problem statement: there may be a physical model, a data file, a target observable, and several possible numerical methods. Before asking for code, it is usually better to ask Gemini to help formulate the workflow.

A good first prompt asks for a plan:

> I want to write a small scientific Python project for simulating a driven two-level quantum system. Before writing code, outline the workflow. Include the physical assumptions, the main functions, tests with known answers, and plots that should be generated.

This prompt is useful because it delays implementation. It gives the student an opportunity to check the model before any code is written. A stronger version includes constraints:

> Use NumPy, SciPy, Matplotlib, and pytest only. Do not use Qiskit or QuTiP. Assume $\hbar = 1$. State the units of time and frequency. Keep the project small enough for a student exercise.

Gemini Chat can also be used as a reviewer. For example:

> Here is my planned model for a driven qubit. Review it before implementation. Focus on hidden assumptions, units, normalization, and what tests would reveal mistakes.

This kind of prompt is often better than asking for code immediately. It asks the tool to identify failure modes. In scientific work, failure modes are central: a wrong convention, missing factor of two, or untested limiting case may be more serious than a syntax error.

Gemini Chat is also useful for converting between levels of description. A student can first write a mathematical description and ask for pseudocode. Then the pseudocode can be moved into the IDE, where Gemini Code Assist can help implement one function at a time.

5 Using Gemini Code Assist in the editor

Inside the editor, Gemini Code Assist is useful because it can use local context. It can suggest completions while the user types, generate code from comments, respond through chat, explain selected code, use selected terminal output, and work with files or folders that are included in the workspace context (Google for Developers, 2026e,d,h).

5.1 Inline code suggestions

Inline suggestions are the most direct form of assistance. The user writes code or comments, and Code Assist proposes the next code continuation. This is useful for short, local tasks. For example:

```
# Return the Pauli X, Y, and Z matrices as complex NumPy arrays.
```

A reasonable suggestion would define the three matrices. This is a good use case because the expected answer is short and easy to verify. Inline suggestions are less appropriate for a complete simulation, a complete report, or a multi-file refactor.

5.2 Code generation from comments

The Code Assist documentation describes prompting in a code file with comments and then generating code from that prompt (Google for Developers, 2026j). In scientific programming, the comment should contain the mathematical intention and the assumptions.

A weak comment is:

```
# evolve the state
```

A stronger comment is:

```
# Evolve a normalized two-level state for one time step dt
# under a constant 2x2 Hermitian Hamiltonian H using expm(-1j * H * dt).
# Return the evolved state and preserve complex dtype.
```

The second comment gives the model, dimensions, numerical method, and type convention. It also makes the generated code easier to review.

5.3 Chat with selected code and terminal output

Gemini Code Assist Chat can use selected code or terminal output. This is particularly useful when debugging. Instead of asking "why does this not work?", select the error message or relevant function and ask:

> The selected function should preserve the norm of a quantum state, but the norm drifts above one after repeated time steps. Explain the likely causes. Focus on Hermiticity, numerical integration, complex dtype, and time-step size. Do not rewrite the code yet.

This is a better debugging prompt because it asks for diagnosis before changes. The user can then request a minimal edit:

> Suggest the smallest change that fixes the most likely issue. Keep the function signature unchanged and do not refactor unrelated code.

5.4 Smart actions and code transformations

The Code Assist documentation lists actions such as generating code, fixing issues, adding documentation, simplifying code, using smart actions, and using code transformation quick fixes (Google for Developers, 2026f,j). These are useful for routine work, especially when the user constrains the scope. For example, `/doc` can be used to add documentation to a function, while `/fix` can be used to address a local error.

For scientific code, a good instruction is always to preserve the mathematical behavior unless the purpose is to change it. For example:

> Add documentation to this function. Do not change the implementation. The docstring should state the Hamiltonian, the units, the expected shape of the state vector, and the returned quantity.

or:

> Simplify this function without changing the equations, constants, default parameters, or output shape. After the edit, summarize the change.

5.5 Finish changes

Code Assist can also complete unfinished code, pseudocode, or `TODO` comments through the documented *Finish changes* workflow (Google for Developers, 2026j). This is useful when the user has already written the structure. For example:

```
def fidelity(psi_target, psi):
    """Return |<psi_target|psi>|^2."""
    # TODO: validate shapes
    # TODO: compute complex inner product
```

```
    # TODO: return squared absolute value as a float
```

This is often better than asking the tool to create a function from nothing. The unfinished code states the intended structure and gives Code Assist less freedom to invent unrelated behavior.

6 Workspace context and context exclusion

One advantage of IDE-integrated assistance is that the tool can use local project context. Google's documentation describes local codebase awareness as a feature that improves relevance through indexing and context from the current open file. It also states that users can specify files and folders in the workspace context, and that an agent can use tools to find and read files it needs (Google for Developers, 2026h,d). This is useful for projects with multiple files, for example when a test file refers to functions in a source module.

However, context should be controlled. The same documentation describes exclusion mechanisms using `.aiexclude` or `.gitignore` files, and notes that exclusion can apply to code generation, code completion, code transformation, and chat (Google for Developers, 2026b). For a scientific project, it is prudent to exclude files that contain secrets, private data, unpublished sensitive results, API keys, credentials, or large raw data files that are not needed for the coding task.

A simple `.aiexclude` file could contain:

```
# Secrets and credentials
.env
apikeys.txt
credentials.json

# Private or sensitive data
data/raw_private/
notebooks/private_notes/

# Large generated outputs
results/raw_simulations/
*.h5
*.npy
```

This does not replace ordinary security practice. API keys should not be committed to Git, and private data should not be placed in prompts. Context exclusion is an additional layer of caution.

7 Agent mode

Gemini Code Assist agent mode is designed for higher-level tasks where the assistant may perform a sequence of steps. Google's documentation describes agent mode as a pair-programming mode in which the user can ask questions, request solutions to complex tasks, generate code, configure tools, and control agent behavior (Google for Developers, 2026a). In VS Code, the documented workflow is to open Gemini Code Assist chat, switch on the Agent toggle, and enter a detailed prompt. The assistant may respond directly or request permission to use a tool.

Agent mode should be used with more caution than ordinary chat. The documentation explicitly warns that there may not be an undo option for changes made to resources outside the IDE, and

that the agent can have access to the file system, terminal actions, and configured tools (Google for Developers, 2026a). In practice, this means that automatic approval should be avoided in course projects unless the user understands exactly what actions the agent is allowed to perform.

A good agent-mode prompt is narrow and verifiable:

> In this repository, add pytest tests for the functions in `src/quantum.py`. Do not modify the source functions. Create tests only for normalization, Hermiticity checks, and the identity-evolution case. After editing, summarize which files changed and which tests should be run.

A broad prompt is riskier:

> Improve this repository.

The second prompt gives too much freedom. In a scientific repository, broad edits can alter equations, constants, data-processing conventions, or plotting logic without making the scientific consequences visible. Agent mode is most useful when the objective is concrete, the files are under version control, and the user is ready to inspect the diff.

8 Prompting strategies for scientific coding

Good prompting reduces ambiguity. The best prompts are not necessarily long, but they make the scientific and programming constraints explicit.

A reliable first move is to ask for assumptions before implementation:

> Before writing code, state the assumptions needed for this implementation. Include assumptions about units, array shapes, normalization, and numerical stability.

A second useful move is to ask for a minimal implementation:

> Write the smallest readable implementation of this function. Use NumPy only. Do not optimize for speed yet. Include a testable example.

A third useful move is to ask for tests before integrating the code:

> Write pytest tests for this function using cases where the answer is known analytically. Include one invalid-input case. Explain what each test checks.

A fourth useful move is to ask for a review rather than a rewrite:

> Review this code for possible scientific errors. Focus on signs, factors of two, units, normalization, array shapes, and validation. Do not rewrite the code yet.

A fifth useful move is to ask for a comparison:

> Compare two ways of implementing this calculation: direct matrix exponentiation and time stepping with a numerical ODE solver. Discuss accuracy, speed, and when each method is appropriate.

These prompts encourage the tool to support reasoning. They also make it easier for the student to detect when a suggestion is inappropriate.

9 Declaring and documenting use

Students should be transparent about the use of Gemini Chat and Gemini Code Assist in a report, repository, or thesis-related workflow. The purpose of declaring AI use is not to discourage the use of tools. The purpose is to distinguish between AI-assisted implementation and human scientific responsibility.

A short declaration in a report could be:

> Gemini Chat and Gemini Code Assist were used as programming assistants during this project. They were used to help plan code structure, generate initial helper functions, suggest tests, review selected code, and draft documentation. All generated code was inspected, modified where necessary, and tested by the author. The tools were not used as sources of scientific conclusions, and the author remains responsible for the correctness of the implementation and interpretation.

For larger projects, it is useful to add an `AI_USE.md` file to the repository. The log does not need to record every accepted inline suggestion. It should record substantial uses, such as generating an initial implementation, debugging a numerical issue, refactoring several files, or drafting tests.

```
# AI use log

Tools:
- Gemini Chat
- Gemini Code Assist in VS Code

Use 1:
Gemini Chat was used to outline the workflow for simulating
Rabi oscillations in a driven two-level system.
Verification: the proposed equations and assumptions were checked manually.

Use 2:
Gemini Code Assist was used to draft helper functions for Pauli matrices,
time evolution, and state fidelity.
Verification: functions were tested on identity evolution and known pi-pulse cases.

Use 3:
Gemini Code Assist Chat was used to review code for possible mistakes involving
complex conjugation, normalization, array shapes, and units.
Verification: suggestions were inspected manually before any changes were made.
```

It can also be useful to document AI assistance in code comments, especially around functions whose first draft was generated by a tool. Such comments should be concise and should not replace technical documentation. For example:

```
def fidelity(psi_target: np.ndarray, psi: np.ndarray) -> float:
    """Return the state fidelity |<psi_target|psi>|^2.

    Implementation note
    -------------------
```

```
    The initial implementation was drafted with Gemini Code Assist
    and then manually reviewed and tested against analytical cases.
    """
    overlap = np.vdot(psi_target, psi)
    return float(np.abs(overlap) ** 2)
```

A declaration should not suggest that Gemini is a co-author. It should simply document the role of the tool. In a course context, the most important statement is that the student inspected and verified the output.

10 Larger example: a quantum-system workflow

This section gives a larger example of how Gemini Chat and Gemini Code Assist can be used together in a scientific workflow. The physical system is a driven two-level quantum system, with Hamiltonian

$$H(t) = \frac{\Delta}{2}\sigma_z + \frac{\Omega(t)}{2}\sigma_x, \tag{1}$$

where Δ is a detuning, $\Omega(t)$ is a drive amplitude, and $\hbar = 1$. The goal is to simulate population transfer from $|0\rangle$ to $|1\rangle$, compute the final-state fidelity with a target state, and compare simple pulse choices.

10.1 Step 1: formulate the problem in Gemini Chat

Begin outside the editor. Ask Gemini Chat to formulate the workflow:

> I want to make a small scientific Python project for a driven two-level quantum system with Hamiltonian
>
> $$H(t) = \frac{\Delta}{2}\sigma_z + \frac{\Omega(t)}{2}\sigma_x.$$
>
> Assume $\hbar = 1$. The goal is to simulate evolution from $|0\rangle$, compute the probability of measuring $|1\rangle$, and compare square pulses with different amplitudes and durations. Propose a minimal workflow, main functions, tests with known answers, and plots. Do not write code yet.

The expected result is not final code. The expected result is a plan: define Pauli matrices, build the Hamiltonian, evolve the state, compute probabilities and fidelities, sweep pulse durations, plot Rabi oscillations, and test limiting cases. The student should check whether the assumptions are correct before moving to the IDE.

10.2 Step 2: create a small file structure

Inside VS Code, create a small project:

```
quantum_gemini_example/
    src/
        quantum_system.py
        plotting.py
```

```
    tests/
        test_quantum_system.py
    run_rabi_sweep.py
    AI_USE.md
    README.md
```

This structure is deliberately simple. It separates the physical model, plotting, tests, and execution script. Gemini Code Assist can now work with a clear context.

10.3 Step 3: implement local functions with Code Assist

In `src/quantum_system.py`, write a comment prompt:

```
# Define Pauli X, Y, Z matrices and computational basis states
# as complex NumPy arrays. Use one-dimensional state vectors.
```

After generating the code, inspect it. The matrices should be Hermitian, σ_y should contain imaginary entries with the correct signs, and the basis states should have shape $(2,)$ if one-dimensional vectors are used consistently.

A plausible implementation is:

```
import numpy as np
from scipy.linalg import expm

SIGMA_X = np.array([[0, 1], [1, 0]], dtype=complex)
SIGMA_Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
SIGMA_Z = np.array([[1, 0], [0, -1]], dtype=complex)

KET_0 = np.array([1, 0], dtype=complex)
KET_1 = np.array([0, 1], dtype=complex)
```

Then ask Code Assist for the Hamiltonian function:

> Write a function `hamiltonian(delta, omega)` that returns the 2 by 2 Hamiltonian $H = (\Delta/2)\sigma_z + (\Omega/2)\sigma_x$. Use the Pauli matrices already defined in this file. Include a docstring stating that $\hbar = 1$.

A concise implementation is:

```
def hamiltonian(delta: float, omega: float) -> np.ndarray:
    """Return H = (delta/2) sigma_z + (omega/2) sigma_x with hbar = 1."""
    return 0.5 * delta * SIGMA_Z + 0.5 * omega * SIGMA_X
```

For time evolution under a constant Hamiltonian, ask:

> Write a function `evolve_constant(psi0, H, duration)` that evolves a state using $U = \exp(-iHT)$. Validate that `psi0` has shape `(2,)` and that `H` has shape `(2,2)`. Return the evolved state as a complex NumPy array.

A possible implementation is:

```
def evolve_constant(psi0: np.ndarray, H: np.ndarray, duration: float) -> np.ndarray:
    """Evolve a two-level state under a constant Hamiltonian."""
```

```
    if psi0.shape != (2,):
        raise ValueError("psi0 must have shape (2,).")
    if H.shape != (2, 2):
        raise ValueError("H must have shape (2, 2).")
    if duration < 0:
        raise ValueError("duration must be non-negative.")

    U = expm(-1j * H * duration)
    return U @ psi0
```

The function should now be tested. It is not enough that it runs.

10.4 Step 4: ask for tests with analytical answers

A good test prompt is:

> Write pytest tests for the quantum functions. Include: identity evolution for zero duration, norm preservation for a Hermitian Hamiltonian, and a resonant pi pulse where $\Delta = 0$, $\Omega = 1$, and $T = \pi$ transfers $|0\rangle$ to $|1\rangle$ up to a phase.

The generated tests may need adjustment, but the scientific content should be clear. For example:

```
import numpy as np

from src.quantum_system import KET_0, KET_1, hamiltonian, evolve_constant


def test_zero_duration_is_identity():
    H = hamiltonian(delta=0.3, omega=1.2)
    psi = evolve_constant(KET_0, H, duration=0.0)
    assert np.allclose(psi, KET_0)


def test_norm_is_preserved():
    H = hamiltonian(delta=0.4, omega=1.0)
    psi = evolve_constant(KET_0, H, duration=2.0)
    assert np.isclose(np.linalg.norm(psi), 1.0)


def test_resonant_pi_pulse_transfers_population():
    H = hamiltonian(delta=0.0, omega=1.0)
    psi = evolve_constant(KET_0, H, duration=np.pi)
    p1 = abs(np.vdot(KET_1, psi)) ** 2
    assert np.isclose(p1, 1.0)
```

This is a good example of responsible use. Code Assist drafts the tests, but the student checks the physics. The π-pulse result follows from the chosen Hamiltonian convention. If the convention changes, the test must change.

10.5 Step 5: use Chat for review

Before building plots, select the file and ask Code Assist Chat:

> Review this file as quantum-simulation code. Focus on Hermiticity, complex conjugation, normalization, units, factors of two, and whether the tests match the Hamiltonian convention. Do not rewrite the code yet.

This prompt is useful because it asks for a scientific code review. The answer should be evaluated manually. The student should not automatically apply every suggestion.

10.6 Step 6: extend to a pulse sweep

Now ask for a small script:

> Write `run_rabi_sweep.py`. Sweep pulse duration from 0 to 4π for $\Delta = 0$, $\Omega = 1$. For each duration, evolve $|0\rangle$, compute $P_1 = |\langle 1|\psi(T)\rangle|^2$, save the data to `results/rabi_sweep.csv`, and save a plot to `results/rabi_sweep.png`. Use pathlib and create the results directory if needed.

The resulting plot should show oscillations. The student should check the location of maxima and minima against the analytical expectation. For this convention, a maximum occurs at $T = \pi$ when $\Delta = 0$ and $\Omega = 1$.

10.7 Step 7: document the result

After running the tests and script, ask Gemini Chat or Code Assist Chat:

> Draft a short README section for this example. Explain the physical model, how to run the tests, how to run the pulse sweep, and what output files are produced. Do not claim that the simulation models decoherence or experimental noise.

The final sentence matters. Generated documentation often overstates the model. Here the model is a coherent two-level system with a constant Hamiltonian during each pulse. It is not a full open-system simulation.

11 Responsible use

Gemini Chat and Gemini Code Assist should be used prudently, especially when they generate more than a few lines of code. Large generated blocks can look coherent while hiding subtle mistakes in logic, units, conventions, numerical methods, or scientific assumptions. In a quantum example, the code may use the wrong sign in a Hamiltonian, mix angular frequency with ordinary frequency, forget a factor of two, fail to preserve normalization, or compare models using an inappropriate validation procedure. The safest workflow is to generate small pieces, read each piece, test it on simple cases, and verify the scientific meaning before moving on.

This caution is part of a broader discussion about AI in science. Messeri and Crockett (2024) argue that AI tools can create *illusions of understanding*: researchers may feel that they understand more than they actually do because a tool produces fluent explanations, plausible code, or apparently objective analyses. They warn that scientific communities may become vulnerable to errors and monocultures if AI tools are adopted as substitutes for understanding

rather than aids to it. The practical implication is that Gemini should be used to support scientific reasoning, not to replace it.

Responsible use also includes privacy and security. Do not paste API keys, passwords, private datasets, confidential manuscripts, or sensitive research material into prompts. Use repository hygiene, `.gitignore`, and `.aiexclude` where appropriate. Review source citations or license warnings when generated code closely matches external sources, since Gemini Code Assist documentation notes that source citations may be provided for suggestions that directly quote at length from a specific source (Google for Developers, 2026f,e). Finally, document substantial AI assistance in the report or repository so that the research workflow remains transparent and reproducible.

12 A compact checklist

Before using Gemini, state the scientific objective. Before generating code, ask for assumptions. Before accepting code, read it. Before using results, run tests. Before reporting conclusions, check whether the code actually implements the intended model.

For Gemini Chat, the most useful prompts are often planning prompts, review prompts, and interpretation prompts. For Gemini Code Assist, the most useful actions are local completions, selected-code explanations, small refactors, tests, documentation, and constrained edits. Agent mode should be reserved for tasks that are well specified, version controlled, and easy to verify.

The central habit is to keep the loop inspectable:

Used this way, Gemini becomes a practical assistant for scientific work: useful for reducing friction, but not a replacement for mathematical understanding, experimental judgment, or reproducible validation.

References

Google Developer Program Help. Activate higher quotas on gemini code assist with gdp premium. https://developers.google.com/profile/help/benefit-gemini-code-assist, 2026. Accessed 2026-07-30.

Google for Developers. Use the gemini code assist agent mode. https://developers.google.com/gemini-code-assist/docs/use-agentic-chat-pair-programmer, 2026a. Accessed 2026-07-30.

Google for Developers. Exclude files from gemini code assist use. https://developers.google.com/gemini-code-assist/docs/create-aiexclude-file, 2026b. Accessed 2026-07-30.

Google for Developers. Sign-in failed. https://developers.google.com/gemini-code-assist/auth/auth_failure_gemini, 2026c. Accessed 2026-07-30.

Google for Developers. Chat with gemini code assist. https://developers.google.com/gemini-code-assist/docs/chat-gemini, 2026d. Accessed 2026-07-30.

Google for Developers. Gemini code assist chat features overview. https://developers.google.com/gemini-code-assist/docs/chat-overview, 2026e. Accessed 2026-07-30.

Google for Developers. Gemini code assist code features overview. https://developers.google.com/gemini-code-assist/docs/code-overview, 2026f. Accessed 2026-07-30.

Google for Developers. Gemini code assist consumer accounts. https://developers.google.com/gemini-code-assist/docs/deprecations/code-assist-individuals, 2026g. Accessed 2026-07-30.

Google for Developers. Configure local codebase awareness. https://developers.google.com/gemini-code-assist/docs/configure-local-codebase-awareness, 2026h. Accessed 2026-07-30.

Google for Developers. Gemini code assist overview. https://developers.google.com/gemini-code-assist/docs/overview, 2026i. Accessed 2026-07-30.

Google for Developers. Code with gemini code assist. https://developers.google.com/gemini-code-assist/docs/write-code-gemini, 2026j. Accessed 2026-07-30.

Google Gemini Apps Help. What you need to sign in to gemini apps. https://support.google.com/gemini/answer/13278668, 2026a. Accessed 2026-07-30.

Google Gemini Apps Help. Use gemini apps. https://support.google.com/gemini/answer/13275745, 2026b. Accessed 2026-07-30.

Lisa Messeri and M. J. Crockett. Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627:49–58, 2024. doi: 10.1038/s41586-024-07146-0.